F20AA - Applied Text Analytics

Coursework 2

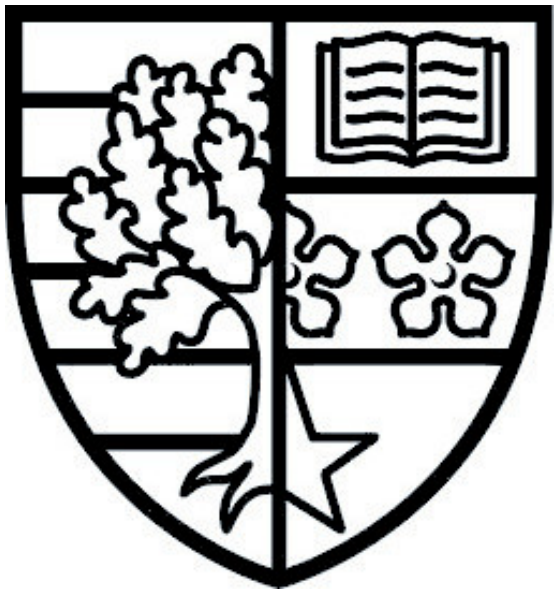
Google Maps Analysis

Group 5:

Mohamed Hafeez Mohamed Ferose - H00416890

Mohamed Ihsan Fazal - H00452069

Sri Sai Vaishnavi Chintha - H00452920



Introduction

This report focuses on building and evaluating a system to predict Google Maps review ratings (1-5 stars) using text data. The task is approached by exploring a range of text classification methods. Initial experiments use TF-IDF representations with traditional machine learning models, followed by sequence-based models such as BiLSTM, and finally transformer-based models including DistilBERT and RoBERTa. This progression allows for a comparison between models that treat text as independent features and those that capture contextual relationships and word order.

In addition to predictive modelling, topic modelling using Latent Dirichlet Allocation (LDA) is applied to analyse differences between low-rated and high-rated reviews. This provides further insight into the types of issues raised in negative reviews and the factors contributing to positive user experiences.

Overall, the aim is to evaluate the effectiveness of different modelling approaches while also gaining a deeper understanding of patterns within the dataset.

Data Exploration and Visualization

The training set contains 288,000 reviews with no missing values. Initial inspection shows a heavy class imbalance: 1-star reviews are the majority class (over 109k), while 5-star reviews are a smaller minority (over 28k). This skewness suggests that F1-Score will be a more reliable metric than raw accuracy. These insights were supported by visualizations including class distribution plots and review length boxplots.

Review Length Analysis:

A word count analysis using boxplots revealed that the median review length stays between 17 and 51 words. 1-star reviews tend to be more “wordy” and descriptive of specific failures, whereas 5-star reviews are often shorter and more punchy.

Keyword Analysis:

Using NLTK to filter stop-words, we identified the most frequent terms at the extremes:

- **1-Star Keywords:** *never, rude, worst, pay, appointment, hours, wait.*
- **5-Star Keywords:** *great, best, amazing, professional, friendly, helpful.*

Observation:

Negative sentiment is heavily tied to temporal frustration (hours, wait) and logistical barriers (pay, appointment), while positive sentiment is driven by superlative adjectives and interpersonal quality (friendly, amazing).

Word count analysis revealed a significant difference in review length. 1-star reviews have a median length of 51 words, nearly three times longer than 4-star reviews (17 words). This suggests dissatisfied customers write detailed, chronological narratives to document their experience, while satisfied customers tend to be brief and superlative.

Technical Insights:

Preprocessing: High frequency of verb variations (call, calling, calls) justifies use of Lemmatization in Step 2.

Modelling: The overlap in vocabulary across ratings (eg. ‘help’ used in both positive and negative contexts) highlights the limitation of frequency-based models and motivates the use of sequence models (LSTMs/Transformers). These models preserve word order, allowing the system to distinguish between ‘helpful

staff' and 'no help at all', which is vital for multi-class accuracy. These observations also justify the use of TF-IDF to better capture discriminative terms rather than relying solely on frequency.

Text Processing and Normalization

Text processing and normalization techniques were applied to the raw review text to prepare it for effective analysis by the models. Basic text cleaning/preprocessing was performed since the reviews in the dataset contained informal text with noise such as inconsistent casing, punctuation, emojis and redundant words. This was followed by a few standard processing and normalization methods such as tokenization, stop-word removal, stemming, and lemmatization.

Basic Cleaning:

Lowercasing: All text was converted to lowercase to ensure consistency across the dataset, so that words such as "Good" and "good" are treated as the same token. This reduces redundancy and improves model efficiency, although it may remove any emphasis conveyed through capitalization.

Punctuation Removal: Punctuations were removed to reduce noise and simplify the text for traditional machine learning models. This improves feature extraction for models such as TF-IDF and Bag of Words. However, it may also remove helpful information, as punctuation can express sentiment (eg. !!!!).

Text Processing:

Tokenization: The cleaned text was split into individual words (tokens), enabling further processing such as stop-word removal, stemming, and lemmatization. Tokenization is essential for converting text into a structured format that models can process.

Stop-word Removal: Common words such as "the", "is", and "and" were removed to reduce noise and focus on more meaningful terms. While this improves efficiency and reduces dimensionality, it may also remove important words such as "not", which can affect sentiment interpretation. Therefore, during preprocessing, the word "not" was explicitly included to preserve important sentiment information.

Text Normalization:

Stemming: Stemming was applied to reduce words to their root form (eg. "studies" → "studi"), helping group similar words and reduce vocabulary size. This improves generalization but may produce non-dictionary words, reducing interpretability.

Lemmatization: Lemmatization converts words to their base dictionary form (eg. "running" → "run"), preserving semantic meaning better than stemming. Although it is computationally more expensive, it produces more meaningful and interpretable text representations.

Both stemming and lemmatization were explored to evaluate their impact on performance. Stemming is faster and reduces dimensionality effectively, but may distort words, while lemmatization provides more accurate and meaningful representations at a slightly higher computational cost. In this dataset, lemmatization produced cleaner and more interpretable text.

Model-Specific Processing:

Text preprocessing significantly improved the quality and consistency of the dataset. While normalization techniques reduce noise and dimensionality, they may also remove useful contextual information. Therefore, different preprocessing strategies were applied depending on the model type.

For traditional machine learning models (eg. TF-IDF, Bag of Words), full preprocessing was applied, including punctuation removal, stop-word removal, and stemming/lemmatization, as these models benefit from reduced noise and a smaller vocabulary. This processing was performed on the training dataset to create the csv file 'train_processed', which was used for the ML models.

In contrast, for deep learning and transformer-based models (eg. LSTM and BERT), only minimal preprocessing was applied (lowercasing and excess whitespace removal). This is because such models rely on contextual information and word order, and aggressive preprocessing such as removing punctuation or altering words, can distort meaning and negatively impact performance. This preprocessing was performed on the training dataset to create the csv file 'train_model', which was used for the deep learning models.

Analysis of review length showed that all the reviews fall below 800 words, most of them being less than 200 words. This indicates differing levels of detail in user reviews, which may influence model performance. Additionally, frequently occurring words such as "time", "get", and "place" reflect general language in user reviews rather than meaningful discriminative features. Since frequency alone does not capture importance, advanced feature representations such as TF-IDF were also used.

Vector Space Model and Feature Representation

Since text data cannot be directly used by ML models, it is converted into numerical representations. Multiple vector space models were explored, including binary representation, Bag of Words (BoW), TF-IDF, n-grams, and word embeddings. These approaches differ in how they represent word importance and contextual information.

Feature Representation Techniques:

Binary Representation: Encodes whether a word is present or not in a document. This simplifies the feature space and is computationally efficient, but it ignores how frequently words appear, potentially losing important information.

Bag of Words (BoW): Represents text by counting word frequencies. This captures how often words appear in a review, which is useful for sentiment classification. However, it treats all words equally and ignores context and word order.

TF-IDF: TF-IDF improves upon BoW by assigning higher importance to words that are frequent in a document but rare across the dataset. This helps highlight more informative and discriminative words, making it more effective for classification tasks.

N-gram Features:

To capture contextual information, n-gram features such as Unigrams, Bigrams and Trigrams were introduced.

Representation	Features
Unigram	138,760
Bigram	3,027,444
Trigram	9,699,147

Including n-grams allows the models to capture important contextual phrases such as “not good” or “very bad”, which improves sentiment understanding. However, higher n-grams dramatically increase feature dimensionality, leading to higher computational cost and potential overfitting.

Feature Importance (TF-IDF Analysis):

The most important words based on TF-IDF scores included terms such as “service”, “good”, “great”, “place”, “staff”. These words reflect key aspects of user reviews and demonstrate that TF-IDF effectively highlights meaningful and discriminative features.

Word Embeddings (Word2Vec):

Word embeddings were also explored using Word2Vec, which represents words as dense vectors.

For example, “good” was closely related to words like “great”, “nice”, “excellent”, and “bad” was associated with words like “terrible”, “horrible”, “poor”.

This demonstrates that Word2Vec captures semantic similarity and contextual meaning, which is not possible with BoW or TF-IDF. However, embeddings require more complex models and are more computationally expensive than traditional representations.

Therefore, different representations offer different advantages:

- Binary and BoW are simple but do not capture context
- TF-IDF improves feature importance and performance
- N-grams capture context but increase dimensionality
- Word embeddings capture semantic meaning but require more complex models.

Vector space representation plays a crucial role in text classification. While simpler models such as BoW and binary representation are efficient, TF-IDF with n-grams capture word importance and context, providing a good balance between performance and complexity. Word embeddings further enhance representation by capturing semantic relationships, making them suitable for more advanced deep learning approaches.

Model training, hyperparameter tuning and evaluation

Model Selection:

The choice of models was guided by the structure of the dataset and the feature representation used. Since the reviews were transformed using TF-IDF, the resulting feature space is high-dimensional and sparse. Linear models are well-suited to such representations, as they can efficiently operate on sparse feature vectors and have been widely used in text classification tasks involving similar data. [1]

Based on this, models such as Logistic Regression, Linear SVC, SGD Classifier, and Ridge Classifier were selected. In addition, Naive Bayes was included as a baseline due to its simplicity and effectiveness in text classification. It has been widely used in sentiment and rating prediction tasks on review datasets, making it a suitable benchmark for comparison. [2]

Experimental Setup:

All models were implemented using a consistent pipeline consisting of TF-IDF vectorization followed by a classifier. The TF-IDF representation was used to convert textual reviews into numerical feature vectors, enabling the application of machine learning algorithms. To evaluate model performance and ensure fair

comparison, hyperparameter tuning was performed using GridSearchCV with a 3-fold cross-validation strategy to balance reliable performance estimation with computational efficiency, given the large size of the dataset.

Two evaluation metrics were used:

- **Accuracy**
- **Macro F1-score**

The macro-F1 score was selected as the primary metric for model selection, as it accounts for class imbalance by giving equal importance to all classes. For each model, both the classifier parameters and feature representation parameters were tuned. Specifically, the TF-IDF vectorizer was varied using different n-gram ranges (unigram and bigram) and maximum feature sizes (10,000, 20,000, and 50,000). These variations were included to analyze the effect of feature representation on model performance.

Model-specific hyperparameters were also explored. For Naive Bayes, the smoothing parameter (α) was tuned. Logistic regression and Linear SVC were tuned using different regularization strengths (C). The SGD Classifier was tuned using different learning rates (α), and the Ridge Classifier was tuned using different regularization parameters. The best parameter combinations for each model were selected based on the highest macro-F1 score obtained from cross-validation.

Results:

Table 1 presents the performance of all models using the best hyperparameter configurations for lemmatized representations and Table 2 for stemming representations. Across all models, Logistic Regression achieved the best overall performance, with a macro-F1 score of 0.5108 and accuracy of 0.6568 for lemmatized text, and similar performance for stemmed text (F1 = 0.5102, accuracy = 0.6561). Naive Bayes provided moderate performance, achieving an F1 score of approximately 0.46, while the SGD Classifier performed the worst among the evaluated models, with an F1 score close to 0.40 despite comparable accuracy. Additionally, the use of bigram features ((1,2)) consistently appeared in the best-performing configurations across all models, indicating that incorporating limited contextual information improves classification performance.

Model runs for cross validation 3 folds on text_lemma:

Model	Best Params	Accuracy	F1 Score
Naive Bayes	{'clf__alpha': 0.1, 'tfidf__max_features': 50000, 'tfidf__ngram_range': (1, 2)}	0.6364	0.4658
Logistic Reg max_iter(1000)	{'clf__C': 3, 'tfidf__max_features': 20000, 'tfidf__ngram_range': (1, 2)}	0.6568	0.5108
SVM	{'clf__C': 2, 'tfidf__max_features': 20000, 'tfidf__ngram_range': (1, 2)}	0.6503	0.4910
SGDC	{'clf__alpha': 0.0001, 'tfidf__max_features': 10000, 'tfidf__ngram_range': (1, 2)}	0.6342	0.4028
Ridge Classifier	{'clf__alpha': 1.0, 'tfidf__max_features': 50000, 'tfidf__ngram_range': (1, 2)}	0.6501	0.4772

Model runs for cross validation 3 folds on text_stem :

Model	Best Params	Accuracy	F1 Score
Naive Bayes	{'clf__alpha': 0.1, 'tfidf__max_features': 50000, 'tfidf__ngram_range': (1, 2)}	0.6344	0.4624
Logistic Reg max_iter(1000)	{'clf__C': 3, 'tfidf__max_features': 20000, 'tfidf__ngram_range': (1, 2)}	0.6561	0.5102
SVM	{'clf__C': 0.5, 'tfidf__max_features': 50000, 'tfidf__ngram_range': (1, 2)}	0.6497	0.4891
SGDC	{'clf__alpha': 0.0001, 'tfidf__max_features': 10000, 'tfidf__ngram_range': (1, 2)}	0.6338	0.4036
Ridge Classifier	{'clf__alpha': 1.0, 'tfidf__max_features': 50000, 'tfidf__ngram_range': (1, 2)}	0.6490	0.4752

Analysis:

The results indicate that linear models, particularly Logistic Regression, are highly effective for this task. This can be attributed to the high-dimensional and sparse nature of TF-IDF features, where linear decision boundaries are sufficient to separate classes effectively.
Although several models achieved similar accuracy scores, there were noticeable differences in macro-F1 scores. This suggests that some models were biased towards majority classes, while Logistic Regression provided a better balance across all classes. This highlights the importance of using macro-F1 as the primary evaluation metric rather than accuracy alone. The relatively lower performance of Naive Bayes may be due to its strong independence assumptions, which limit its ability to capture relationships between words. Similarly, the SGD Classifier showed lower F1 performance, likely due to its sensitivity to hyperparameters and convergence behavior.
The consistent selection of bigram features across models suggests that incorporating short-range context improves the representation of review text. Finally, the comparison between lemmatized and stemmed text shows minimal differences in performance, suggesting that both preprocessing approaches produce similar representations for this task.

Modelling text as a Sequence

The models discussed in Section 4 treat text as a bag-of-words representation, where word order and contextual relationships are not considered. While such representations are effective, they fail to capture the sequential nature of language.
To address this, sequence-based models such as Bidirectional LSTM (BiLSTM) and transformer-based models such as DistilBERT and RoBERTa were explored. These models process text as ordered sequences and use contextual information to improve representation of meaning. Prior work has shown that such models are effective for sentiment and rating prediction tasks on review datasets. [3,4]

BiLSTM

Architecture and Training:

A Bidirectional LSTM (BiLSTM) model was implemented to capture contextual dependencies in text. The input text was first tokenized using a vocabulary size of 10,000 and converted into padded sequences of fixed length (160 tokens).
An embedding layer was used to learn dense vector representations of words, followed by a Bidirectional LSTM layer to capture both forward and backward context. A dropout layer was applied to reduce overfitting, followed by a dense layer with ReLU and a softmax output layer for multi-class classification.
Hyperparameters including LSTM units, embedding dimension, batch size, and number of epochs were tuned using 3-fold cross-validation. The best configuration used 128 LSTM units, embedding dimension of 200, batch size of 64, and 5 epochs. A more complex BiLSTM model improved training accuracy but did not yield significant gains in validation performance, indicating overfitting and limited generalization benefits.

Results and Analysis:

The BiLSTM model achieved an accuracy of 0.6597 and a macro-F1 score of 0.5359, outperforming all classical models from Section 4. This improvement suggests that modelling text as a sequence provides additional benefits over bag-of-words representations.
However, the performance gain over Logistic Regression (F1 ≈ 0.51) is relatively small, indicating that TF-IDF-based models already capture substantial information. Additionally, training time was significantly higher (~8 hours), limiting the extent of hyperparameter exploration. More extensive tuning or longer training could potentially improve performance further.

Model	Best Params	Accuracy	F1 Score
BiLSTM	{'batch_size': 64, 'epochs': 5, 'model__embedding_dim': 200, 'model__lstm_units': 128}	0.6597	0.5359

Transformer-based Models

Architecture and Training:

Transformer-based models, specifically DistilBERT and RoBERTa, were implemented to further improve performance using pre-trained contextual embeddings. Unlike LSTM models, these architectures use self-attention mechanisms to process the entire sequence simultaneously, allowing more effective modelling of long-range dependencies.
The models were fine-tuned on the train dataset using tokenized text inputs with varying maximum sequence lengths. Key hyperparameters such as learning rate, number of epochs, and sequence length were tuned experimentally. Learning rates of 2e - 5 and 3e - 5, sequence lengths between 128 and 200 tokens, and 2-3 training epochs were explored.

Results and Analysis:

DistilBERT achieved a maximum macro-F1 score of 0.6031 with an accuracy of 0.6567 using a sequence length of 160, learning rate of 2e - 5, and 2 training epochs. RoBERTa further improved performance, achieving the highest overall results with a macro-F1 score of 0.6159 and accuracy of 0.6703 under similar training settings (sequence length = 160, learning rate = 2e - 5, 2 epochs).

These results significantly outperform both the BiLSTM model (F1 ≈ 0.536) and the best classical model (F1 ≈ 0.51).

Distilbert-base-uncased vectorizer:

Max_len	learning rate	epochs	accuracy	f1
128	2e-5	2	0.6459	0.5932
160	2e-5	2	0.6567	0.6031
160	2e-5	3	0.6516	0.5999
160	3e-5	2	0.6555	0.6032
160	3e-5	3	0.6459	0.5888
200	3e-5	2	0.6528	0.5952

Roberta-base vectorizer:

Max_len	learning rate	epochs	accuracy	f1
256	2e-5	2	0.6034	0.5522
160	2e-5	2	0.6703	0.6159
160	3e-5	2	0.6717	0.6107

The superior performance of transformer-based models can be attributed to their use of pre-trained contextual embeddings and attention mechanisms, which enable a deeper understanding of semantic relationships in text. Unlike TF-IDF or LSTM-based approaches, transformers can capture both local and global context more effectively.

The results show a clear improvement from classical models to BiLSTM, and further to transformer-based models, indicating that increasingly sophisticated representations lead to better performance. In particular, RoBERTa demonstrates the strongest performance, suggesting that more advanced pre-training strategies contribute to improved classification accuracy.

Kaggle Submissions

Based on cross-validation results, the best-performing hyperparameters for each model were selected for final training and Kaggle submission. Logistic Regression using lemmatized text achieved a score of 61.7 (with 20,000 TF-IDF features), slightly outperforming the 10,000 feature configuration (61.5). The BiLSTM model improved performance to 62.7, while DistilBERT achieved a marginal increase to 62.9. RoBERTa produced the best result

with a score of 66.6, outperforming all other models. These results are consistent with earlier findings, where transformer-based models demonstrated superior performance.
Overall, a clear progression in performance is observed from classical machine learning models to sequence models and finally to transformer-based approaches. While Logistic Regression provided competitive results with minimal computational cost, more advanced models such as RoBERTa achieved significantly higher performance by capturing deeper contextual relationships in text. However, these improvements come with increased computational complexity, highlighting a trade-off between efficiency and accuracy.

Topic Modelling

We applied Latent Dirichlet Allocation (LDA) to 5,000 samples each of 1-star and 5-star reviews. Using the text_lemma feature and a 15-topic configuration, we identified the distinct thematic drivers of customer sentiment.

- **1-Star (Functional Friction):** Topics are “event-driven” and industry-specific. Key themes include Property/Rent disputes (Topic 1), Medical/Hospital delays (Topics 6, 10), Financial/Billing errors (Topics 3, 8, 15), and Automotive Repairs (Topic 9). Staff rudeness (Topic 14) is a universal driver of low ratings.
- **5-Star (Experiential Satisfaction):** Topics are “sentiment-driven” and focus on Professionalism (Topics 4, 7) and Interpersonal Excellence (Topics 3, 15). Keywords like “friendly”, “helpful”, and “amazing” dominate, showing that high ratings stem from emotional satisfaction.

Key Observations and Inferences:

Based on the extracted topics, we inferred the following regarding consumer language:

1. **Specificity vs. Generality:** 1-star reviews describe concrete failures (eg: insurance, prescriptions, leases). 5-star reviews use broad, superlative praise (eg: “best”, “highly recommend”) applicable across all sectors.
2. **The "Time" Variable:** In negative reviews (Topic 7), “hour” and “wait” represent a resource lost. In positive reviews, time is linked to efficiency and professional consistency.
3. **Informed Conclusion:** 1-star ratings are triggered by objective process failures (billing, long waits), whereas 5-star ratings are driven by subjective staff attitude and overall experience.

1-Star Topic Distribution (Negative Topics)

5-Star Topic Distribution (Positive Topics)

The Context Problem:

A key finding was that words like “help” and “recommend” appear in both 1-star and 5-star topics. In negative reviews, these are often negated (e.g., “did not help”), which Bag-of-Words and LDA cannot capture. This highlights a key limitation of these approaches, as they fail to account for word order and context. As a result, sequence models are better for this task, explaining their improved performance over traditional classifiers.

Conclusion

Multiple approaches to text classification we explored, including traditional vector space models and sequence-based deep learning methods. Preprocessing techniques such as normalization and stop-word handling were shown to significantly impact feature quality and model performance. TF-IDF with n-grams provided a strong balance between simplicity and effectiveness, while sequence models such as LSTM and BERT demonstrated improved ability to capture contextual meaning. The results highlight that no single approach is universally optimal, and that performance depends on the choice of representation, preprocessing, and model architecture. Overall, combining appropriate preprocessing with context-aware models leads to more accurate and robust sentiment classification.

Link to Google Colab version of python notebook:

https://colab.research.google.com/drive/1ybNOVbFYMc2oTsGwLIrGqR3SkFVPllzM?usp=sharing

Task Distribution

Team Member	Sections	Tasks Performed
Mohamed Hafeez	1 and 6	Conducted Exploratory Data Analysis (EDA) and visualization; Performed LDA-based topic modelling on 1-star and 5-star reviews to analyse differences in themes driving negative and positive feedback.
Mohamed Ihsan	2 and 3	Implemented the text preprocessing pipeline, including lemmatization and stop-word removal; performed vectorization and feature representation via BoW, TF-IDF, n-grams and Word Embeddings.
Sri Sai Vaishnavi	4 and 5	Led the predictive modeling phase; benchmarked traditional classifiers against deep learning architectures (LSTMs/Transformers) and handled hyperparameter tuning for multi-class accuracy.

References

1. Liu, Z., 2020.Yelp Review Rating Prediction: Machine Learning and Deep Learning Models.
2. Puh, K. and Bagić Babac, M., 2023. Predicting sentiment and rating of tourist reviews using machine learning. Journal of hospitality and tourism insights, 6(3), pp.1188-1204.
3. Kusal, S., Patil, S., Gupta, A., Saple, H., Jaiswal, D., Deshpande, V. and Kotecha, K., 2023, August. Sentiment analysis of product reviews using deep learning and transformer models: a comparative study. In International Conference on Artificial Intelligence on Textile and Apparel (pp. 183-204). Singapore: Springer Nature Singapore.
4. Areshey, A. and Mathkour, H., 2024. Exploring transformer models for sentiment classification: A comparison of BERT, RoBERTa, ALBERT, DistilBERT, and XLNet. Expert Systems, 41(11), p.e13701.